Sentiment Analysis Report

4.1 A Look at the Dataset

For this project, I used the **Datafiniti Amazon Consumer Reviews of Amazon Products (May 2019)** dataset, which I found on Kaggle. It's a substantial dataset containing **28,332 rows and 24 columns**, covering a wide range of products including batteries, tablets, Fire TV sticks, and Kindles.
The primary feature I analysed was the 'reviews.text' column, which contains the raw consumer feedback. Since the dataset was clean, I didn't need to drop any rows due to missing data.

4.2 Preprocessing Steps

To prepare the raw text for Natural Language Processing (NLP), I followed these steps:

- **Feature Selection** — I isolated the 'reviews.text' column using index selection to use as my primary feature variable.
- **Data Cleaning** — I utilized dropna() to ensure the spaCy pipeline wouldn't encounter Null values during processing.
- **Normalisation** — I converted all text to lowercase and stripped leading/trailing whitespace to maintain consistency for the model.
- **Tokenization** — I utilized spaCy's 'en_core_web_md' model to perform tokenization, breaking each review down into individual linguistic units.
- **Stop-word Filtering** — I removed "stop words" (common terms like "the" and "of") to ensure the model focuses on semantically significant tokens rather than grammatical noise.

4.3 Checking the Results

I tested the sentiment analysis function on a sample of reviews to evaluate model performance. The polarity scores reflect the emotional intensity, while the labels indicate the general sentiment:

-

Review	Polarity	Label
"one of the items is bad quality..."	−0.4500	Negative
"bulk is always the less expensive way..."	−0.5000	Negative
"for the price I am happy"	+0.8000	Positive
"equal if not superior to name brand"	+0.4723	Positive
"I definitely love the price and quantity"	+0.3500	Positive

When comparing two battery reviews using cosine similarity, the model returned a score of **0.78**. This indicates that the model successfully identified the semantic similarity between the two reviews, recognizing they discussed the same category of product.

4.4 The Good and the Not-So-Good

What worked well

- **Unsupervised Learning** — Because TextBlob relies on a pre-defined sentiment lexicon, it performed effective analysis immediately without requiring manual training or labelled examples.
- **Metric Granularity** — The model provides nuanced metrics, including polarity (emotional valence) and subjectivity (factual vs. opinion-based text).
- **Computational Efficiency** — The pipeline processed the large dataset quickly, demonstrating high throughput.
- **Noise Reduction** — By implementing stop-word filtering, the model maintained focus on high-information tokens.

Where there's room for improvement

- **Lack of Contextual Awareness** — The model struggles with sarcasm and nuanced language. For example, a neutral comment about bulk buying was incorrectly flagged as negative.

- **Stop-word Risks** — Aggressive stop-word filtering can sometimes strip essential negations; for instance, "not good" could lose its negative semantic force if "not" is removed.
- **Domain Sensitivity** — Since TextBlob is a general-purpose tool, it may occasionally misinterpret domain-specific technical jargon found in product reviews.
- **Label Sensitivity** — The binary nature of the labeling logic (polarity > 0 or < 0) means that neutral or near-neutral sentiments are often forced into Positive or Negative categories.